

Quantum Operations

Quantum State Preparation

Before running a quantum algorithm, we need to decide the starting state of your qubits. This is called quantum state preparation.

Default starting state in PennyLane

In PennyLane, every qubit automatically starts in the $|0\rangle$ state.

- In vector form $\Rightarrow [1, 0]$
- If we are satisfied with the starting state, no changes are required.

Setting a custom starting state

If the requirement is a specific state:

```
| qml.StartPrep
```

- Example $\Rightarrow$ 2 qubits

```
| dev = qml.device('default.qubit', wires=2)

| @qml.anode(dev)
| def circuit(state=None):
|     qml.StatePrep(state, wires=range(2))
|     return qml.state()

| state = circuit([1/2, 1/2, 1/2, 1/2])
```

- This prepares the two qubits in the state:

```
| [0.5, 0.5, 0.5, 0.5]
```

Important: The state must be normalized

A quantum state must be normalized.

The sum of the squared magnitudes = 1

If the state isn't normalized, PennyLane can fix it:

```
| qml.StatePrep(state, wires=range(2), normalize=True)
```

Easier option for simple states → `qml.BasisState`

To prepare qubits in basic 0/1 configuration:

```
| qml.BasisState
```

- Example → 3 qubits in the $|100\rangle$ state
 - We can write

```
| qml.BasisState(np.array([1, 0, 0]), wires=range(3))
```

- Alternative, use its position number

```
| qml.BasisState(np.array(4), wires=range(3))
```

- Why 4? ⇒ Because $|100\rangle$ is the 4th state in binary counting.

```
| |000> = 0
| |001> = 1
| |010> = 2
| |011> = 3
| |100> = 4
```

Single Qubit Gates

Now that we know how to prepare quantum states, let's apply unitary operation (quantum gates).

Starting with single-qubit gates, which act on just one qubit at a time.

Since, we have known how to use:

- Pauli gates
- Hadamard gates
- Rotation gates

PennyLane also provides many other built-in gates that can be used.

Taking the table below as reference for most common single-qubit gates.

Gate	Matrix	PennyLane function
X	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$	<code>qml.PauliX</code> or <code>qml.X</code>
H	$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$	<code>qml.Hadamard</code>
Z	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$	<code>qml.PauliZ</code> or <code>qml.Z</code>
S	$\begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$	<code>qml.S</code>
T	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$	<code>qml.T</code>
Y	$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$	<code>qml.PauliY</code> or <code>qml.Y</code>
RZ	$\begin{pmatrix} e^{-i\frac{\theta}{2}} & 0 \\ 0 & e^{i\frac{\theta}{2}} \end{pmatrix}$	<code>qml.RZ</code>
RX	$\begin{pmatrix} \cos(\frac{\theta}{2}) & -i \sin(\frac{\theta}{2}) \\ -i \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{pmatrix}$	<code>qml.RX</code>
RY	$\begin{pmatrix} \cos(\frac{\theta}{2}) & -\sin(\frac{\theta}{2}) \\ \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{pmatrix}$	<code>qml.RY</code>

Multi-Qubit Gates

To build interesting quantum algorithms, we also need multi-qubit gates.

We've already learned how to use CNOT gate in PennyLane, which is technically the only multi-qubit gate we really need.

However, PennyLane includes many other multi-qubit gates as well.

Using the table as a reference:

Gate	Matrix	PennyLane function
$CNOT$	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$	<code>qml.CNOT</code>
CZ	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$	<code>qml.CZ</code>
CRZ	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & e^{-i\frac{\theta}{2}} & 0 \\ 0 & 0 & 0 & e^{i\frac{\theta}{2}} \end{pmatrix}$	<code>qml.CRZ</code>
CRX	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \cos\left(\frac{\theta}{2}\right) & -i\sin\left(\frac{\theta}{2}\right) \\ 0 & 0 & -i\sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}$	<code>qml.CRX</code>
CRY	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ 0 & 0 & \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}$	<code>qml.CRY</code>
CU	$\begin{pmatrix} I_2 & 0 \\ 0 & U \end{pmatrix}$	<code>qml.ctrl</code> or <code>qml.ControlledQubitUnitary</code>

<i>SWAP</i>	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$	qml.SWAP
<i>TOF</i>	$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$	qml.Toffoli
<i>CCZ</i>	$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \end{pmatrix}$	qml.CCZ

If the unitary required isn't one of PennyLane's built-in gates, we can apply it using:

```
| qml.QubitUnitary
```

- This allows to apply any unitary matrix, as long as we know its matrix.
- **HOWEVER**, this is not recommended for real quantum hardware, because actual quantum devices only supports a specific set of basic (universal) gates.

Example

- U is a single-qubit applied to wire 0
- V is a two-qubit binary applied to wires [0, 1]

```
| dev = qml.device("default.qubit", wires=2)
```

```
@qml.qnode(dev):
def qubit_unitaries(U, V):
    qml.QubitUnitary(U, wires=0)
    qml.QubitUnitary(V, wires= [0, 1])
    return qml.state()
```

- After defining matrices U (2×2) and V (4×4), running:

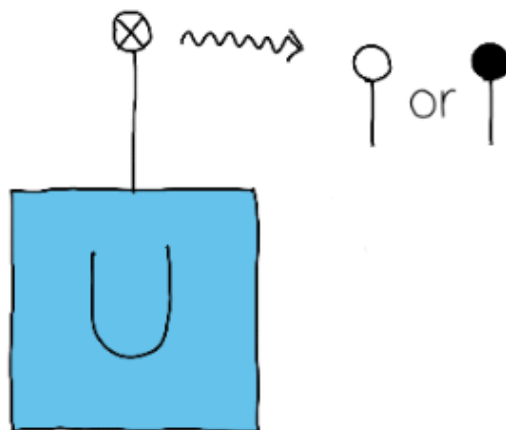
```
print(qubit_unitaries(U, V))
```

Produces:

```
[ 0.08606318+0.28698854j 0.59055748-0.06504235j
 0.63856782-0.16439135j -0.34016643+0.08207004j]
```

Controlled Unitaries

The most common multi-qubit gates are controlled gates.



In a controlled gate:

- A block dot on a wire $\Rightarrow$ Qubit is a control with "control_value = 1"
- A clear dot on wire $\Rightarrow$ Qubit is a control with "control_value = 0"

The gate is only applied when:

- Wires with "control_value = 1" are in the state $|1\rangle$
- Wires with "control_value = 0" are in the state $|0\rangle$

If the gate already exists in PennyLane, like `qml.RX`, `qml.PauliX`, etc. we can use:

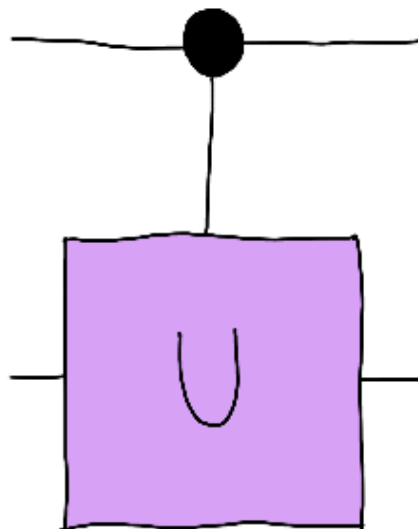
| `qml.ctrl`

- `op` → The PennyLane operator like `qml.RX` or a quantum function
- `control` → The control wires
- `control_values` → The required control values for each control wire

`qml.ctrl` → Acts as a wrapper around the gate.

Example

We apply an RX gate to wire 1, controlled by wire 0 with control value 1



```
dev = qml.device("default.qubit", wires = 2)

@qml.qnode(dev)
def controlled_gate_circuit(angle):
    qml.PauliX(0)
    qml.ctrl(qml.RX, control=(0), control_values=(1))(angle, wires=[1])
    return qml.state()
```

Running:

```
| print(controlled_gate_circuit(0.5))
```

Outputs:

```
| [0.+0.j 0.+0.j 0.96891242+0.j 0.-0.24740396j]
```

- The *op* argument of *qml.ctrl* can also be a user-defined quantum function.

If the matrix of unitary is given:

```
| qml.ControlledQubitUnitary
```

- It requires:
 - $U \rightarrow$ Unitary matrix
 - *control_wires* $\rightarrow$ List of control wires
 - *wires* $\rightarrow$ Wires where the unitary acts
 - *controlled_values* $\rightarrow$ Required controlled values
- Unlike *qml.ctrl*, this is not a wrapper function.

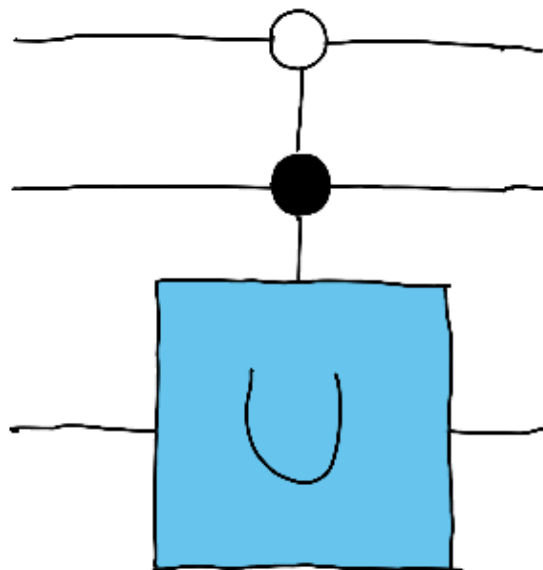
Example

Apply the unitary:

```
| U = [[ 0.94877869, 0.31594146],  
       [-0.31594146, 0.94877869]]
```

to wire 2, controlled by:

- wire 0 with control value 0
- wire 1 with control value 1




```
dev = qml.device("default.qubit", wires = 3)
```

```
U = [[ 0.94877869, 0.31594146],  
      [-0.31594146, 0.94877869]]
```

```
@qml.qnode(dev)  
def circuit_controlled_unitary():  
    qml.PauliX(wires = 1)  
    qml.ControlledQubitUnitary(  
        U,  
        control_wires = (0, 1)  
        wire = (2),  
        control_values = (0,1)  
    )  
    return qml.state()
```

Running:

```
print(circuit_controlled_unitary())
```

Outputs:

```
[ 0.+0.j 0.+0.j 0.94877869+0.j -0.31594146+0.j  
 0.+0.j 0.+0.j 0.+0.j 0.+0.j]
```

Inverse Operations

In many quantum algorithms, we need to apply the inverse of operator U .

If U is unitary, its inverse is the adjoint, written as $U^\dagger$.

In PennyLane, we can apply the adjoint using:

```
qml.adjoint
```

This works on:

- A single quantum gate
- A quantum function

If we apply it to a gate like `qml.RX`, we can use it in two equivalent ways:

```
| qml.adjoint(qml.RX(0.5, wires=0))
```

OR

```
| qml.adjoint(qml.RX)(0.5, wires=0)
```

Both produce the same result.

If we apply on a quantum function, it must be used as a wrapper.

For example:

```
| def q_function(theta, phi, omega):  
    qml.RX(theta, wires=0)  
    qml.RX(phi, wires=1)  
    qml.RX(omega, wires=2)
```

To apply its adjoint:

```
| qml.adjoint(q_function)(theta, phi, omega)
```

Codercise PF.2.1 - Quantum state preparation

Using `qml.StatePrep` write the `state_preparation` QNode that prepares a quantum state proportional to

$$|\psi\rangle = \alpha|001\rangle + \beta|010\rangle + \gamma|100\rangle.$$

Note that `state_preparation` takes the complex numbers α (alpha), β (beta), and γ (gamma) as arguments.

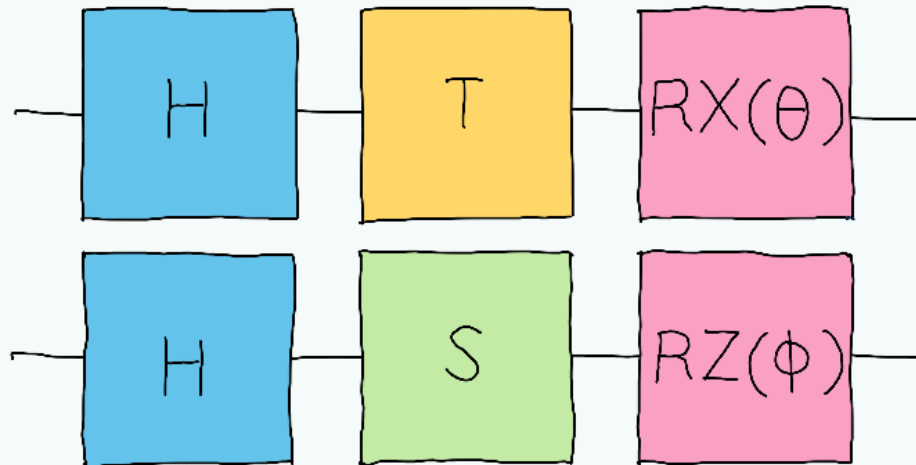
Do **not** assume that this state is normalized. That is, it *might* be the case that

$$|\alpha|^2 + |\beta|^2 + |\gamma|^2 \neq 1.$$

```
1 dev = qml.device("default.qubit", wires=3)
2
3 @qml.qnode(dev)
4 def prep_circuit(alpha, beta, gamma):
5     """
6     Prepares the state  $\alpha|001\rangle + \beta|010\rangle + \gamma|100\rangle$ .
7     """
8     # 1. Create the full state vector for 3 qubits (8 elements)
9     # The indices for  $|001\rangle$ ,  $|010\rangle$ , and  $|100\rangle$  are 1, 2, and 4 respectively.
10    state_vector = np.array([0, alpha, beta, 0, gamma, 0, 0, 0])
11
12    # 2. Normalize the state vector manually
13    norm = np.sqrt(np.sum(np.abs(state_vector)**2))
14    state_vector = state_vector / norm
15
16    # 3. Use qml.StatePrep to prepare the state on all 3 wires
17    qml.StatePrep(state_vector, wires=[0, 1, 2])
18
19    return qml.state()
20
21 # Test the function
22 alpha, beta, gamma = 1/np.sqrt(3), 1/np.sqrt(3), 1/np.sqrt(3)
23 print("The prepared state is", prep_circuit(alpha, beta, gamma))
```

Codercise PF.2.2 - A circuit with a single qubit gate

Using the table shown on the Theory side, complete the `single_qubit_gates` QNode so that it applies the circuit shown below and returns the quantum state.



The `single_qubit_gates` circuit depends on the parameters `theta` for the RX gate and `phi` for the RZ gate.

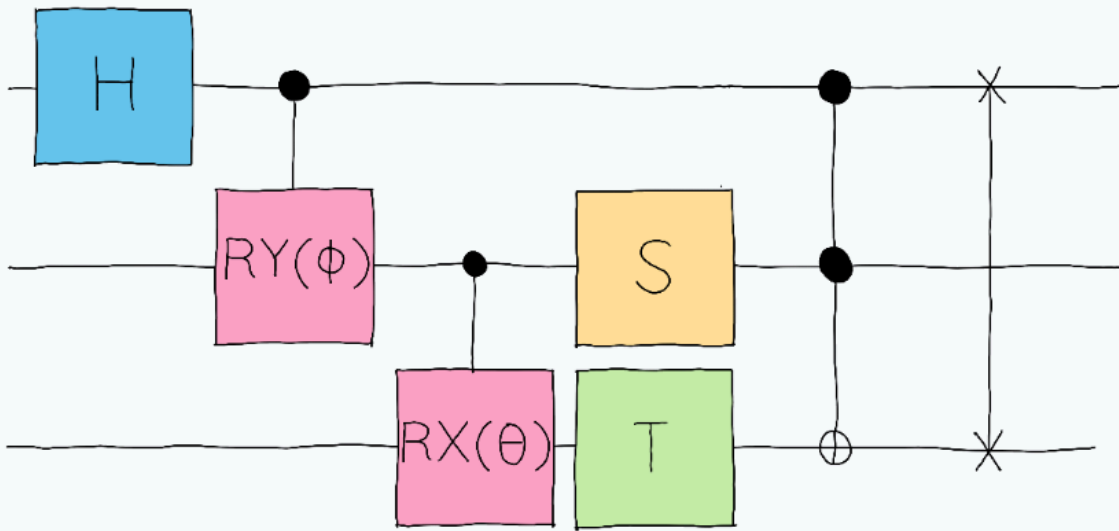
```

1 dev = qml.device("default.qubit", wires=2)
2
3 @qml.qnode(dev)
4 def single_qubit_gates(theta, phi):
5     """
6     Implements the quantum circuit shown in the statement
7     Args:
8         - theta, phi (float): The arguments for the RX and RZ gates, respectively
9     Returns:
10        - (np.array): The output quantum state.
11    """
12
13    #####
14    # YOUR CODE HERE #
15    #####
16
17    # Apply gates to Wire 0
18    qml.Hadamard(wires=0)
19    qml.T(wires=0)
20    qml.RX(theta, wires=0)
21
22    # Apply gates to Wire 1
23    qml.Hadamard(wires=1)
24    qml.S(wires=1)
25    qml.RZ(phi, wires=1)
26
27    return qml.state()
28
29 theta, phi = np.pi/3, np.pi/4
30 print("The output state of the circuit is: ", single_qubit_gates(theta, phi))

```

Codercise PF.2.3 - A circuit with multi qubit-gate

Using the table shown on the theory side, complete the `multi_qubit_gates` QNode so that it applies the circuit shown below and returns the quantum state.



Note that the `multi_qubit_gates` circuit depends on the parameters `theta` and `phi` for the `CRX` and the `CRY` gates, respectively.

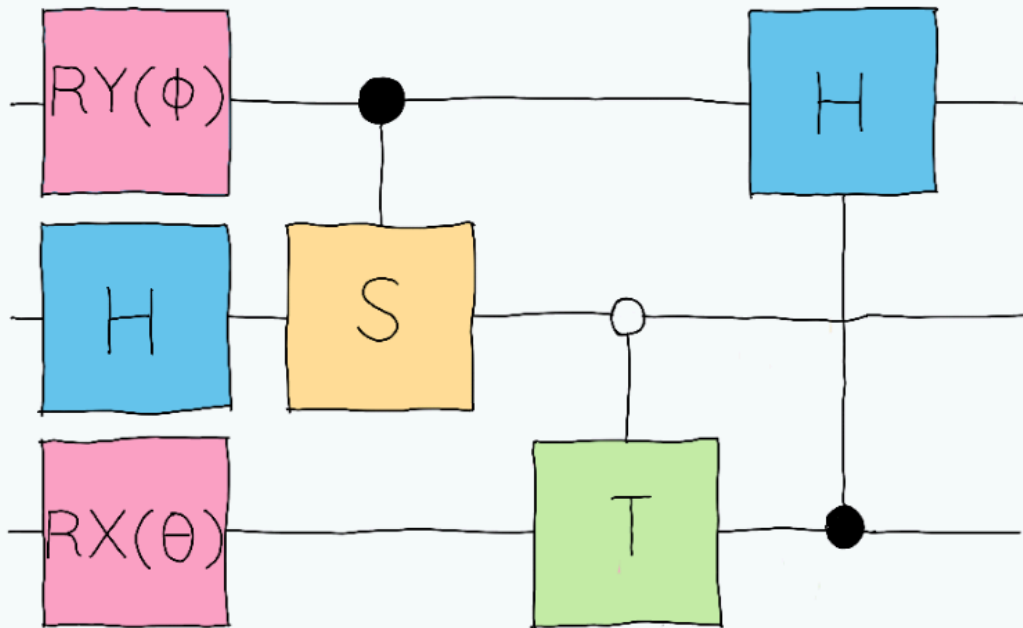
```

1 dev = qml.device("default.qubit", wires=3)
2
3 @qml.qnode(dev)
4 def multi_qubit_gates(theta, phi):
5     """
6     Applies the circuit shown in the figure above
7     """
8     # Apply operations in order
9     qml.Hadamard(wires=0)
10    qml.CRY(phi, wires=[0, 1])
11    qml.CRX(theta, wires=[1, 2])
12    qml.S(wires=1)
13    qml.T(wires=2)
14    qml.Toffoli(wires=[0, 1, 2])
15    qml.SWAP(wires=[0, 2])
16
17    return qml.state()
18
19 theta, phi = np.pi/3, np.pi/4
20 print("The output state is: \n", multi_qubit_gates(theta, phi))

```

Codercise PF.2.4 - Gates under control

Complete the following QNode to implement the following circuit containing controlled native PennyLane gates. Remember that you can use `qml.ctrl`!



Note that the `multi_qubit_gates` circuit depends on the parameters `theta` and `phi` for the RX and the RY gates, respectively.

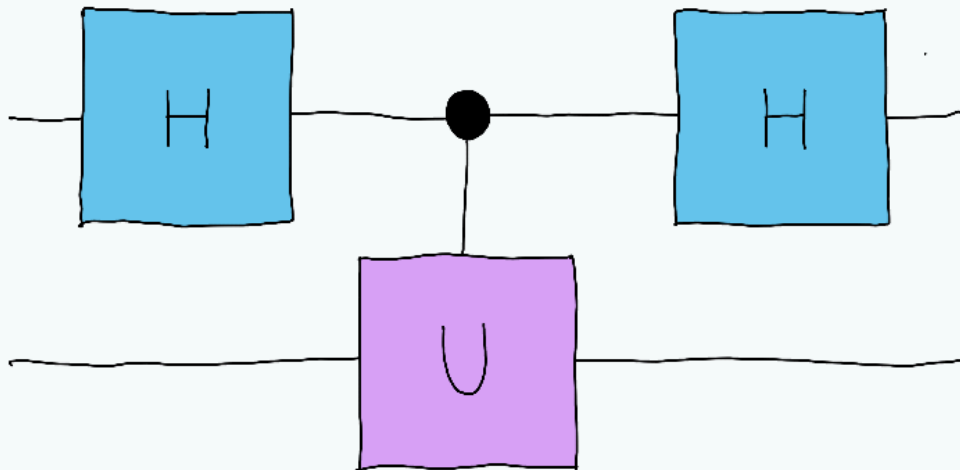
```

1 dev = qml.device("default.qubit", wires=3)
2
3 @qml.qnode(dev)
4 def ctrl_circuit(theta, phi):
5     """
6     Implements the circuit shown in the Codercise statement using qml.ctrl
7     """
8     # Step 1: Base gates
9     qml.RY(phi, wires=0)
10    qml.Hadamard(wires=1)
11    qml.RX(theta, wires=2)
12
13    # Step 2: S gate on wire 1 controlled by wire 0 (control-on-one)
14    qml.ctrl(qml.S, control=0)(wires=1)
15
16    # Step 3: T gate on wire 2 controlled by wire 1 (control-on-zero)
17    qml.ctrl(qml.T, control=1, control_values=[0])(wires=2)
18
19    # Step 4: Hadamard on wire 0 controlled by wire 2 (control-on-one)
20    qml.ctrl(qml.Hadamard, control=2)(wires=0)
21
22    return qml.state()
23
24 # Example execution
25 theta, phi = 0.1, 0.2
26 print("Output state:\n", ctrl_circuit(theta, phi))

```

Codercise PF.2.5 - Kicking U back

One building block of many quantum algorithms is the phase kickback routine, shown below.



Write the `phase_kickback` QNode, which applies the phase kickback algorithm to a single-qubit operator represented by a given matrix. This time, you may need `qml.ControlledQubitUnitary`.

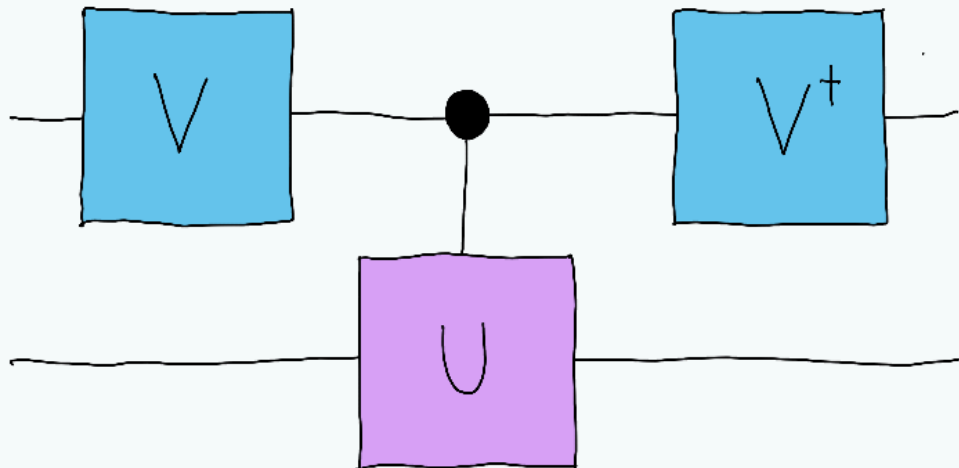
```

1 dev = qml.device("default.qubit", wires=2)
2
3 @qml.qnode(dev)
4 def phase_kickback(matrix):
5     # Apply Hadamard to the control qubit (wire 0)
6     qml.Hadamard(wires=0)
7
8     # Use the matrix, then the control wire, then the target wire
9     qml.ctrl(qml.QubitUnitary, control=0)(matrix, wires=1)
10
11     # Apply the final Hadamard to the control qubit
12     qml.Hadamard(wires=0)
13
14     return qml.state()
15
16 # Example matrix from the problem statement
17 matrix = np.array([[ -0.69165024-0.50339329j,  0.28335369-0.43350413j],
18                    [ 0.1525734 -0.4949106j , -0.82910055-0.2106588j ]])
19
20 print("The state after phase kickback is: \n", phase_kickback(matrix))

```

Codercise PF.2.6 - Do, Apply, Undo

Many quantum algorithms follow a pattern shown in the figure below.



We apply an operator V , then a controlled operation U , and finally the inverse $V^\dagger$.

In the code below, the single-qubit operation V is given to you as the subcircuit `do`, which depends on a parameters k . It prepares a state proportional to $|0\rangle + k|1\rangle$.

You are also provided U as the `apply` subcircuit, which simply applies the `theta` dependent gate `IsingXX`. Check out the [PennyLane documentation](#) for more details!

Complete the `do_apply_undo` circuit below that applies the pattern discussed above, with $V = \text{do}(k)$ and $U = \text{apply}(\text{theta})$.

```

1 dev = qml.device("default.qubit")
2
3 def do(k):
4     qml.StatePrep([1,k], wires = [0], normalize = True)
5
6
7 def apply(theta):
8
9     qml.IsingXX(theta, wires = [1,2])
10
11 @qml.qnode(dev)
12 def do_apply_undo(k, theta):
13     # Step 1: V = do(k)
14     do(k)
15
16     # Step 2: Controlled-U. The subcircuit 'apply' is controlled by wire 0.
17     qml.ctrl(apply, control=0)(theta)
18
19     # Step 3: V_dagger = adjoint(do(k))
20     qml.adjoint(do)(k)
21
22     return qml.state()
23
24 # Test with provided parameters
25 k, theta = 0.5, 0.8
26 print("The output state is: \n", do_apply_undo(k, theta))

```